

SegmentX

A Multi-Method Image Segmentation and Object Extraction Toolkit

Course: CSE3010 Computer Vision

Project Report

Student Name	Mustafa
Registration Number	25BCE11232
Academic Year	2026

Prepared in alignment with the supplied VITyarthi Build Your Own Project instructions and CSE3010 syllabus.

1. Introduction

Computer Vision systems often require segmentation before meaningful objects or regions can be analyzed. Segmentation separates an image into regions with shared properties such as intensity, color, or boundary structure. SegmentX was designed as a single command-line toolkit that places several classical segmentation strategies behind a common workflow and continues the result into object extraction and evaluation.

The project is aligned with CSE3010 topics covering low-level image processing, edge detection, image segmentation, clustering and object analysis. The supplied course outline specifically lists edge-based approaches, region growing, K-Means, Mean-Shift and object detection among its topics and experiments.

The implementation is intentionally modular. Each major operation lives in a separate Python module and the pipeline coordinates them. This makes the project easier to test, compare, extend and execute from a terminal.

2. Problem Statement

Image segmentation techniques have different assumptions and failure modes. A color-clustering method may work well where objects are visually distinct, while seeded region growing depends strongly on the selected seed, and edge-based segmentation depends on the quality and closure of boundaries. In teaching and experimentation, these methods are frequently implemented as separate demonstrations. SegmentX provides a common workflow in which the same input can be processed by multiple segmentation methods and then converted into actual object-level outputs.

3. Objectives

- Implement a reproducible image preprocessing workflow.
- Implement multiple classical segmentation approaches behind one CLI.
- Convert segmentation results into cleaned masks and connected objects.
- Export object crops, bounding boxes, centroids and area measurements.
- Compare algorithms using processing time, foreground coverage and object count.
- Support optional binary ground-truth evaluation using IoU, Dice and pixel accuracy.
- Provide automated tests and design documentation suitable for command-line evaluation.

4. Functional Requirements

- **FR1 - Input and preprocessing:** validate an input image, resize oversized images, denoise and prepare grayscale/color representations.
- **FR2 - Segmentation:** support K-Means, Region Growing, Edge Based, Mean-Shift Based and the boundary-aware Hybrid mode.
- **FR3 - Object extraction:** clean masks, find connected components and export object crops with metadata.
- **FR4 - Comparison/evaluation:** report runtime, object count and foreground ratio; compute IoU, Dice and pixel accuracy when a ground-truth mask is provided.
- **FR5 - CLI:** expose the main workflows through command-line arguments without requiring a GUI.
- **FR6 - Persistence:** save images, JSON metadata and CSV comparisons under a configurable output directory.

5. Non-Functional Requirements

- **Performance:** cap very large working images to a configurable maximum resolution.
- **Reliability:** validate file paths and parameter ranges and return clear command-line errors.
- **Maintainability:** isolate preprocessing, segmentation, extraction, evaluation and visualization logic.

- **Usability:** document all main commands and expose help through argparse.
- **Reproducibility:** use deterministic random seeds for K-Means-based operations in the demo workflow.
- **Testability:** cover core behaviour with pytest unit tests.

6. System Architecture

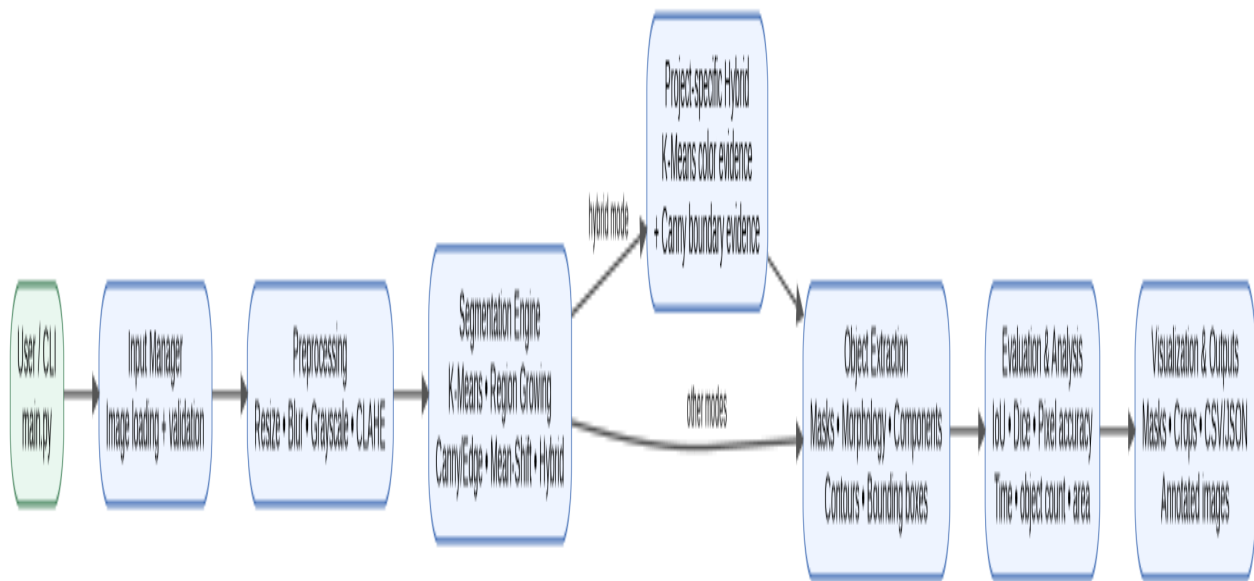


Figure 1. SegmentX system architecture.

The architecture separates the command-line interface from the computational modules. The pipeline receives the validated image, performs preprocessing, dispatches the selected segmentation method, sends the resulting mask to object extraction, and finally records measurements and output artifacts.

7. Process Workflow

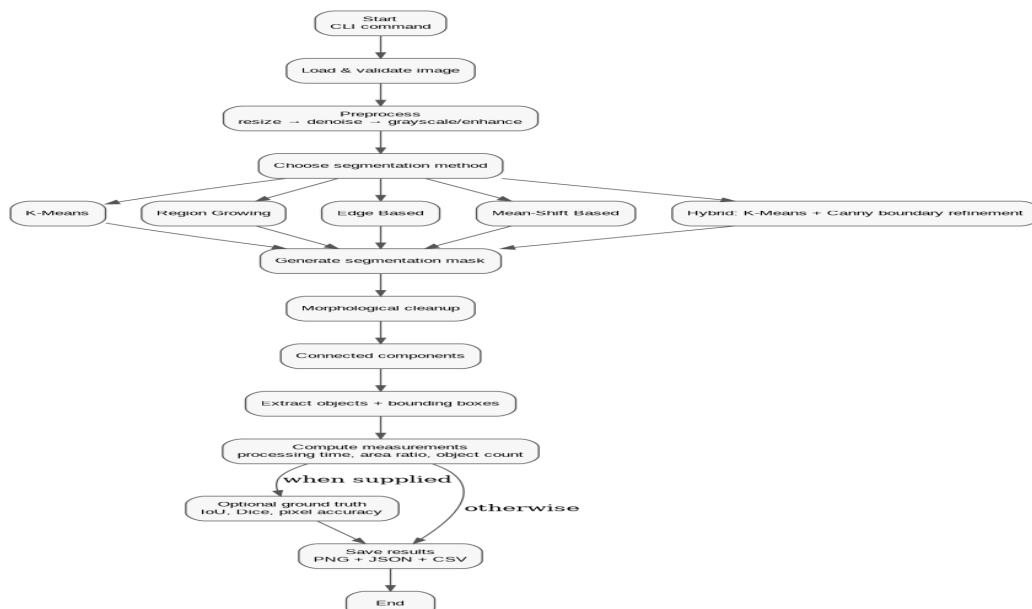


Figure 2. End-to-end processing workflow.

8. Design Diagrams

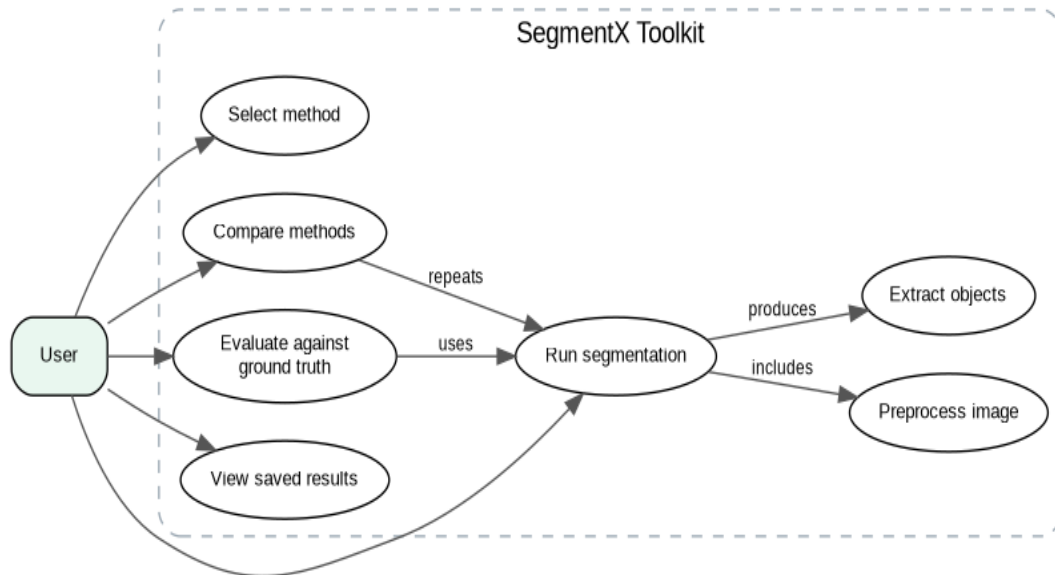


Figure 3. Use case view.

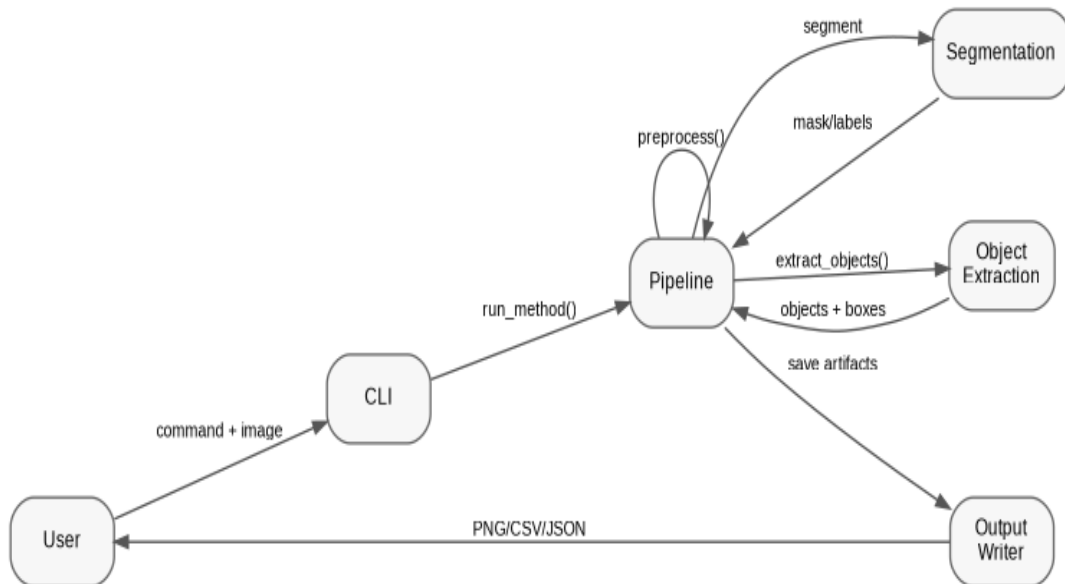


Figure 4. Sequence of a typical CLI run.

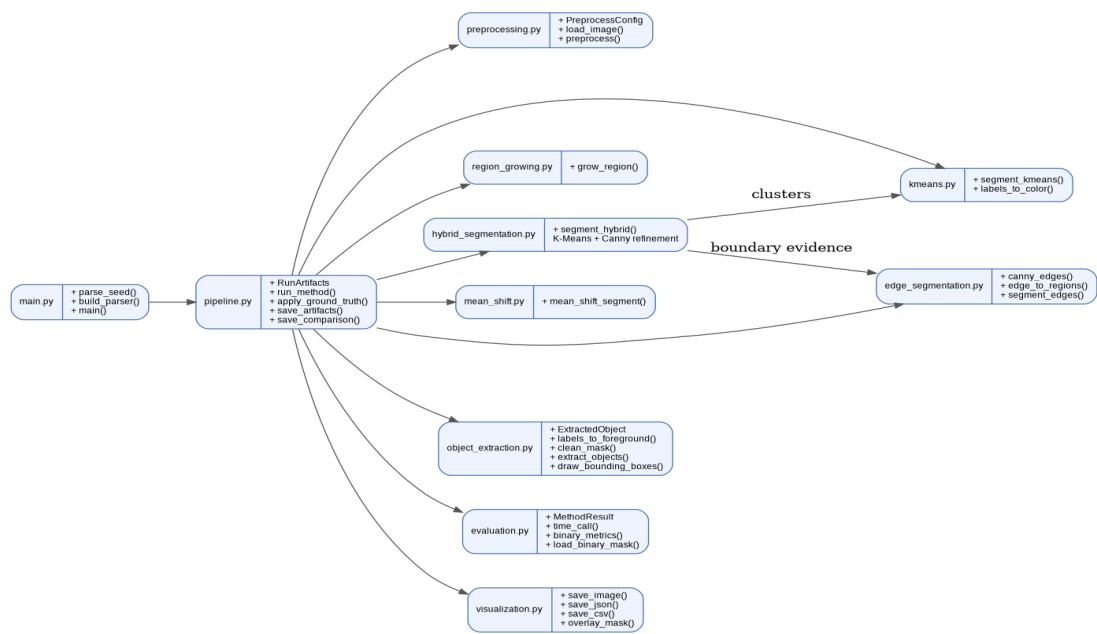


Figure 5. Implementation module/class relationships.

9. Segmentation Methods

9.1 K-Means color segmentation

Pixels are converted to CIELAB color space and grouped into a configurable number of clusters using OpenCV K-Means. A region-label image is produced. For object extraction, the dominant label touching the image border is treated as background and the remaining labels are merged into a candidate foreground mask.

9.2 Seeded Region Growing

A seed point is supplied as x,y coordinates. A queue-based breadth-first expansion adds neighbouring pixels whose intensity differs from the evolving region mean by no more than the configured threshold. This method is intentionally exposed with the seed parameter because the seed is an important part of its behaviour.

9.3 Edge-based segmentation

Canny edge detection produces a binary boundary map. Morphological closing attempts to bridge small gaps, after which sufficiently large external contours are filled to form candidate foreground regions.

9.4 Mean-Shift based segmentation

OpenCV pyramid mean-shift filtering is used to shift image colors toward local spatial/color modes. The filtered image is then labeled by a compact K-Means step so explicit regions are available to the common object-extraction stage. The project therefore uses a hybrid Mean-Shift + labeling design for predictable runtime and a consistent downstream interface.

10. Object Extraction

The selected segmentation mask is normalized to binary form and cleaned with morphological opening/closing. Connected components are then measured. Components below a configurable image-area ratio are rejected. Retained components receive an object ID, area, bounding box and centroid. Crops are written with pixels outside each object set to black so each extracted object can be inspected independently.

11. Evaluation Methodology

The toolkit records processing time, object count and foreground ratio for every method. When a binary ground-truth mask is supplied, three additional measures are calculated: intersection over union (IoU), Dice coefficient and pixel accuracy. These metrics operate on the foreground/background definition used by the reference mask. A comparison without ground truth therefore reports measurable execution characteristics rather than inventing quality scores.

12. Demo Dataset and Results

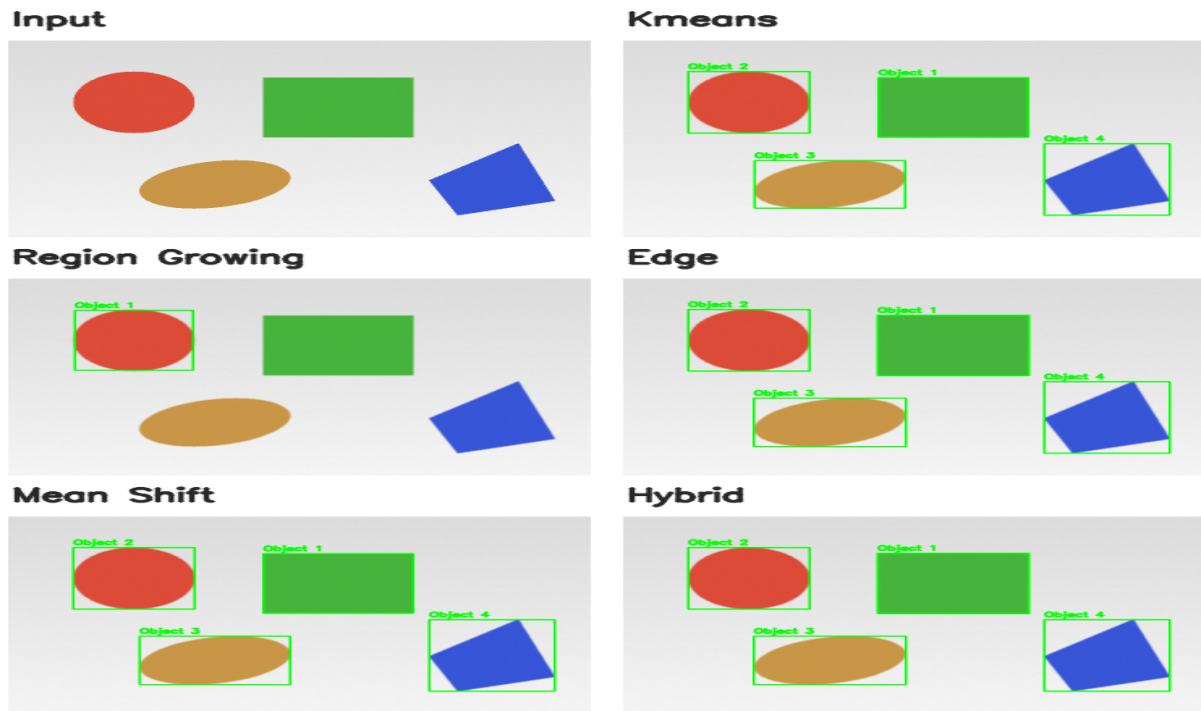


Figure 6. Deterministic demo image and annotated outputs for the five supported methods.

Method	Time (s)	Objects	Foreground	IoU	Dice	Pixel Acc.
K-Means	0.4465	4	0.223	0.9964	0.9982	0.9992
Region Growing	0.0439	1	0.050	0.2232	0.3649	0.8267
Edge Based	0.0058	4	0.226	0.9862	0.9931	0.9969
Mean-Shift Based	0.5121	4	0.223	0.9985	0.9992	0.9997
Hybrid	0.2818	4	0.226	0.9862	0.9931	0.9969

Table 1. Results from the included deterministic synthetic demo. Values depend on hardware and runtime conditions; they are included as a reproducible project demonstration rather than as a benchmark claim.

The demo image contains four colored geometric objects on a light background and a binary reference mask. The K-Means, Mean-Shift based and Hybrid modes recover four objects with foreground coverage close to the reference. The Edge-based method also produces four objects with similar coverage on this particular demo. Region Growing identifies the seeded red object only, illustrating the intended seed dependence and why method configuration is part of the experiment. The Hybrid mode is deliberately included as a separate experiment to study the interaction between color-region and boundary evidence.

13. Implementation Details

The repository is organized around small focused modules. ``main.py`` parses commands and handles user-facing errors. ``pipeline.py`` coordinates execution and artifact saving. ``preprocessing.py`` handles image preparation. The five segmentation modules each expose a focused callable, with ``hybrid_segmentation.py`` composing the K-Means and Canny components for the project-specific experiment. ``object_extraction.py`` manages morphological cleanup, connected components and object metadata. ``evaluation.py`` contains metrics and runtime helpers. ``visualization.py`` provides file-writing utilities.

The project also includes unit tests for preprocessing, segmentation, object extraction and metric calculation. A synthetic data generator keeps the demonstration reproducible and avoids requiring a large external dataset simply to verify that the code runs.

14. Repository Structure

```
segmentx/ main.py statement.md README.md requirements.txt pyproject.toml src/ preprocessing.py kmeans.py
region_growing.py edge_segmentation.py mean_shift.py object_extraction.py evaluation.py visualization.py
pipeline.py tests/ scripts/ data/ outputs/ diagrams/ report/
```

15. Testing Approach

- Preprocessing test verifies required representations and compatible dimensions.
- K-Means test verifies shape, integer labels and multiple regions.
- Region Growing test verifies that the supplied seed is included and the region is non-empty.
- Canny test verifies that a boundary is detected on a synthetic object.
- Mean-Shift test verifies filtered-image and label dimensions.
- Hybrid segmentation test verifies binary-mask output and color visualization dimensions.
- Object extraction test verifies connected-component metadata and crop creation.
- Evaluation test verifies perfect IoU, Dice and pixel accuracy for identical masks.

The complete suite is run with ``pytest`` from the repository root.

16. Challenges Faced

- Different segmentation algorithms naturally produce different forms of output, so a common mask-to-object interface was required.
- Region Growing is inherently seed dependent and therefore cannot be treated like a fully automatic global method without additional logic.
- Edge maps are not automatically object masks; contour closure and morphological processing are needed before connected components can be used reliably.
- Mean-Shift filtering by itself does not expose a compact integer region map, so an explicit labeling stage was added for common downstream processing.
- Automatic foreground/background inference from border labels is an assumption and can fail on images where the foreground occupies image borders.

17. Learnings and Key Takeaways

- Segmentation is not a single problem with a universally correct algorithm; method assumptions matter.
- A useful Computer Vision application needs preprocessing, representation, segmentation, post-processing and evaluation, not just a model call.

- A common data interface makes otherwise different algorithms composable and testable.
- Quantitative metrics should be tied to a clearly defined reference mask and not interpreted without understanding what the metric measures.
- Command-line reproducibility and modular code make experimentation easier to repeat and evaluate.

18. Future Enhancements

- Add interactive seed selection while preserving the existing CLI mode.
- Add Graph-Cut or additional segmentation methods for advanced experimentation.
- Add texture descriptors and SIFT/HOG-based object characterization after segmentation.
- Support batch processing of image folders and automatic aggregation of results.
- Add a richer HTML experiment report generated from saved JSON/CSV artifacts.
- Add configurable background-selection strategies and explicit foreground/background labels.

19. References

- VITyarthi - Build Your Own Project, General Project Instructions & Submission Guidelines, supplied project instruction PDF.
- CSE3010 Computer Vision Course, supplied course syllabus PDF.
- OpenCV Python API used by the implementation for image I/O, filtering, Canny, K-Means, mean-shift filtering and connected components.

20. Conclusion

SegmentX provides a complete, command-line Computer Vision workflow centered on image segmentation and object extraction. The project integrates multiple segmentation approaches with common post-processing, object-level outputs, evaluation metrics, tests, design diagrams and reproducible demo data. Its modular architecture also leaves clear extension points for advanced Computer Vision techniques.